

UNIVERSIDAD DE MANIZALES

Tecnología en desarrollo de software para negocios digitales

Seguridad de aplicaciones

Universidad de manizales

Sergio Pinilla Valencia

Pablo Nieto Pérez

Negocioseguro (compra segura)

Tarea 5, fundamentos de cifrado (simétrico, asimétrico y hashing)

Documento comparativo

31 de octubre de 2026

1. Introducción

En el producto 2 dejé tapada la inyección sql de negocioseguro pero en las limitaciones quedó anotado que las claves de los usuarios seguían guardadas en texto plano, o sea que si alguien llegaba a ver la tabla usuarios veía todas las contraseñas tal cual, esta tarea es el paso antes de corregir eso, la idea es entender bien cómo funcionan las herramientas de criptografía que trae php antes de meterlas en la aplicación

Trabajé con los códigos base que el profe compartió en la clase, uno de cifrado simétrico con aes, uno de cifrado asimétrico con rsa y uno de hashing con sal, los dos primeros los corrí tal cual para ver cómo se comportan y a partir del de aes y el de hash armé los dos archivos que pide la tarea, equipo_cifrado.php y equipo_hash_sal.php, con datos inventados por mí, todo lo corrí en mi xampp local así que no hay ningún dato real de por medio

El documento tiene la preparación del entorno, lo que salió al correr cada script (los mensajes originales contra los descifrados y los hashes), la tabla comparativa y el reto de aplicación con los tres escenarios de negocioseguro, al final van la bibliografía y el código de los dos archivos

2. Objetivos

2.1 Objetivo general

Probar en mi entorno local cómo funcionan el cifrado simétrico, el cifrado asimétrico y el hashing con sal en php para poder escoger con argumentos cuál usar en cada parte de negocioseguro

2.2 Objetivos específicos

1. Cifrar con aes-256-cbc un dato inventado y comprobar que al descifrarlo vuelve exactamente igual

2. Mostrar con dos usuarios que tienen la misma contraseña por qué un hash sin sal es peligroso y cómo la sal lo corrige

3. Comparar la reversibilidad, la velocidad y el manejo de claves de las herramientas con números que medí en mi computador

4. Decidir qué mecanismo va en cada uno de los tres escenarios de negocioseguro y justificarlo

3. Desarrollo

3.1 Preparación del entorno

Prendí apache desde el panel de xampp y creé la carpeta tareas/tarea5 dentro del proyecto compraseguras (c:\xampp\htdocs\compraseguras\tareas\tarea5), ahí quedaron los dos archivos y se abren en http://localhost/compraseguras/tareas/tarea5/, el xampp trae php 8.2.12 con openssl 3.0.11 que es lo que usan las funciones openssl_encrypt y openssl_pkey_new

Las evidencias de este documento las saqué corriendo los dos scripts el 31 de octubre de 2026, como el iv y las sales son aleatorios cada vez que se recarga la página salen valores distintos, eso es lo esperado y lo explico más abajo

3.2 Cifrado simétrico, equipo_cifrado.php

Partí del código base de aes-256-cbc y le cambié tres cosas, la primera fue el mensaje: en vez del backup puse un dato que yo inventé y que se parece a lo que guardaría la tienda, un pedido con el usuario, los últimos números de la tarjeta y la dirección de envío

La segunda fue la clave, el comentario del código base dice que tiene que tener exactamente 32 caracteres pero cuando le saqué el strlen a ClaveSecretaDe32CaracteresExact me dio 31, openssl no avisa nada y la rellena por dentro, entonces el código funciona pero la clave no es la que uno cree, puse una de 32 caracteres justos y el script imprime el largo de las dos para que se vea

La tercera fue agregar dos pruebas que para mí muestran mejor cómo funciona: cifrar el mismo mensaje con otro iv y tratar de descifrar con una clave equivocada, también le puse la medición del tiempo con hrtime para tener el dato de velocidad de la tabla

Lo que salió al correrlo:

Largo de la clave del codigo base: 31 caracteres
Largo de la clave que use: 32 caracteres

Mensaje original: Pedido 1087 de pablo_nieto, tarjeta terminada en 4417, envio a Calle 23 # 21-40 Manizales
IV (en hexadecimal): a98a9df6e344f75a1abf34bfaf9a3630
Mensaje cifrado (en base64):
rfTEII6kj6LXrEcNKP25wMZ7dMSocNCe/L1sBz1mJr9wZ3B86dLPOsq+g5r1yuBqAv80CG08x3CiFEmBu0L6yxOYcs3QtXNkn5JSc9E7ej6sULFiLKv4YzUpmX5zg7hH

Mensaje descifrado: Pedido 1087 de pablo_nieto, tarjeta terminada en 4417, envio a Calle 23 # 21-40 Manizales
¿El descifrado es igual al original? SÍ
Tiempo para cifrar: 21.4 microsegundos, para descifrar: 5.7 microsegundos

=== EL MISMO MENSAJE CON OTRO IV ===
Cifrado con otro IV:
GsP0srkzI9aPwXY/tA36lDEsWMwvJxpGT1sgZJmv6Ewz3hT/ekCfqhHYSuyRZLHvfpWxVjGk6WlVcIRP7InpsxDPGHF+X5DEZs9tvRUWxbfTOkYEiMXugs4JCV5Yxlmh

=== DESCIFRAR CON UNA CLAVE EQUIVOCADA ===
Resultado: falla, no se recupera el mensaje

El descifrado salió idéntico al original y la comparación con === da SÍ, entonces el proceso es completamente reversible, siempre que se tenga la misma clave y el mismo iv, con la clave equivocada openssl_decrypt devuelve false y no hay forma de sacar el mensaje, también se ve que el mismo mensaje con otro iv da un cifrado totalmente diferente, eso sirve para que dos pedidos iguales no se puedan reconocer mirando el texto cifrado, el iv no es secreto y se puede guardar al lado del dato, lo que no puede quedar a la vista es la clave

3.3 Hashing con sal, equipo_hash_sal.php

Acá partí del código base de los dos usuarios con la misma contraseña, cambié los usuarios por dos inventados (laura_gomez y andres_rios) y les puse a los dos la misma clave manizales2026, después le agregué la parte que comprueba un login con la sal guardada y otra parte con password_hash, que es la función que php recomienda para contraseñas y la que pienso usar en la app, por último la medición de tiempo

Lo que salió al correrlo:

Usuario 1: laura_gomez, clave: manizales2026
Usuario 2: andres_rios, clave: manizales2026

=== SIN SAL ===
Hash Usuario 1: 865aab0bdd7222146ec4efaa6a8a836c1b1575fc443ac38c1c2743a5df61f12a
Hash Usuario 2: 865aab0bdd7222146ec4efaa6a8a836c1b1575fc443ac38c1c2743a5df61f12a
¿Son idénticos? SÍ

=== CON SAL ÚNICA POR USUARIO ===
Sal Usuario 1: f6d869575f437c98fa1365e5bc4b19d4
Hash Usuario 1 (con sal):
21832f03aca5e1474e36d25b5cc2aca3a140ea7106ebd7f7bdb8c81fb1ca9beb

Sal Usuario 2: a068f8c4a20d509b3f7aa31c63a1f6a0
Hash Usuario 2 (con sal):
e0e1f2d1e4b47926f13952c21aebf74aace43019a89a91f07820ce3ddf682a90
¿Son idénticos? NO

=== COMPROBAR EL LOGIN CON LA SAL GUARDADA ===
laura_gomez escribe manizales2026: clave correcta
laura_gomez escribe manizales2025: clave incorrecta

=== CON PASSWORD_HASH (BCRYPT, LA SAL VA DENTRO DEL HASH) ===
Hash Usuario 1:
$2y$12$msubKGDVUEg4NyAffpSOxO27CIlaEPMv/s/qWDcdockCQ91coSQGO
Hash Usuario 2: $2y$12$F9bkKICVc4cfgRc1pGZroOqoP9rqbyWNNszgGG8g9MvwVLteKpHdC
¿Son idénticos? NO
password_verify con la clave correcta: true
password_verify con una clave equivocada: false

=== VELOCIDAD ===
sha256 con sal: 2 microsegundos
password_hash con costo 12: 220 milisegundos

Sin sal los dos hashes son idénticos y eso es el problema, si alguien ve la tabla sabe de una que los dos usuarios tienen la misma clave y si la clave es común la puede sacar con una tabla arcoíris (una lista enorme de hashes ya calculados), con la sal de 16 bytes aleatorios cada usuario queda con un hash diferente aunque la clave sea la misma, y la sal no es secreta, se guarda junto al hash porque se necesita para comprobar el login como se ve en la prueba de laura_gomez

Con password_hash pasa lo mismo pero sin tener que manejar la sal a mano, bcrypt la genera y la mete dentro del mismo texto del hash (lo que va después de $2y$12$), y password_verify la saca de ahí para comparar, la diferencia grande está en la velocidad: el sha256 con sal tardó un par de microsegundos y password_hash con costo 12 tardó unos 220 milisegundos, para un usuario que inicia sesión eso no se nota pero para alguien que quiere probar millones de claves sobre un hash robado es la diferencia entre minutos y años, por eso para contraseñas lo lento es bueno

3.4 Cifrado asimétrico, código base de rsa

Este no hacía parte de los archivos que se entregan pero lo corrí para poder llenar la tabla, con el código base generé el par de llaves rsa de 2048 bits, cifré con la llave pública Datos sensibles del pedido #4521 y lo descifré con la privada y salió igual, generar las llaves tardó entre 130 y 220 milisegundos y cada cifrado o descifrado entre 1 y 3 milisegundos, o sea mucho más lento que aes

Además probé cifrar un texto de 300 caracteres y openssl_public_encrypt falló, con una llave de 2048 bits rsa solo deja cifrar unos 245 bytes, por eso en la práctica no se usa para cifrar archivos o datos largos sino para cifrar una clave de aes o para que dos partes se pongan de acuerdo, que es lo que hace https por debajo

3.5 Tabla comparativa

Los tiempos los medí en mi computador con hrtime, son de una sola corrida entonces sirven para comparar entre ellos más que como número exacto

Herramienta	**Tipo**	**¿Es reversible?**	**Velocidad (en mi pc)**	**Manejo de claves**	**Para qué sirve**
aes-256-cbc (openssl_encrypt)	Cifrado simétrico	Sí, con la misma clave y el mismo iv	Muy rápida, unos 20 microsegundos para cifrar y menos de 10 para descifrar	Una sola clave de 32 bytes que tienen que conocer los dos lados, hay que guardarla fuera del código y fuera	Datos guardados que se necesitan leer después: backups, datos de envío

				de la base	
rsa 2048 (openssl_public_encrypt)	Cifrado asimétrico	Sí, pero solo con la llave privada	Lenta, de 1 a 3 milisegundos por operación y unos 200 ms para generar las llaves	Dos llaves, la pública se puede compartir con cualquiera y la privada no sale nunca del dueño	Mandar datos a otro sistema, firmar, intercambiar claves de aes
sha256 sin sal (hash)	Hashing	No	Muy rápida, un par de microsegundos	No usa clave	Comprobar que un archivo no cambió, no sirve para contraseñas
sha256 con sal	Hashing con sal	No	Muy rápida, un par de microsegundos	No usa clave, la sal es aleatoria por usuario y se guarda junto al hash	Evita hashes repetidos, pero por ser tan rápido se puede atacar por fuerza bruta
password_hash (bcrypt, costo 12)	Hashing con sal y lento a propósito	No	Lenta a propósito, unos 220 milisegundos	No usa clave, la sal la genera y la guarda la misma función dentro del hash	Contraseñas de usuarios, se comprueba con password_verify

Lo que más me sirvió de la tabla es ver que la pregunta clave es si el dato se necesita volver a leer, si la respuesta es sí toca cifrar (simétrico o asimétrico según quién lo va a leer) y si la respuesta es no, como pasa con una contraseña que solo hay que comprobar, toca hashear y con un algoritmo lento

3.6 Reto de aplicación, tres escenarios de negocioseguro

Para cada escenario escogí el mecanismo pensando en tres preguntas: si el dato se tiene que volver a leer, quién lo tiene que leer y qué pasa si se roba la base de datos

Escenario 1, las contraseñas de los usuarios

Mecanismo: hashing con sal, con password_hash y password_verify

Justificación: la tienda nunca necesita saber cuál es la clave de un usuario, solo comprobar que la que escribe en el login es la misma, entonces no tiene sentido cifrarla porque cualquier cosa cifrada se puede descifrar si alguien consigue la clave, con el hash aunque se roben la tabla usuarios no hay cómo devolverlo a la contraseña, la sal hace que dos usuarios con la misma clave no queden con el mismo hash (como se vio en la sección 3.3) y bcrypt es lento a propósito para que probar claves por fuerza bruta no sea viable, en la app esto queda en registro.php con password_hash y en login.php con password_verify, que es justo el cambio

que dejé pendiente en el producto 2, y la columna clave tiene que quedar de al menos 60 caracteres que es lo que mide un hash de bcrypt

Escenario 2, la copia de seguridad de la base de datos

Mecanismo: cifrado simétrico con aes-256

Justificación: el backup (negocioseguro_2026.sql) si hay que poder leerlo después porque toca restaurarlo, entonces no puede ser un hash, como lo cifra y lo descifra el mismo administrador no hace falta una llave pública y aes es mucho más rápido, que importa porque un backup es un archivo grande y rsa ni siquiera deja cifrar más de 245 bytes, lo importante acá es el manejo de la clave: no puede quedar en el código ni en la misma carpeta del backup, porque si se roban los dos juntos el cifrado no sirvió para nada, tiene que ir en una variable de entorno o en un gestor de secretos y el iv se genera nuevo en cada backup

Escenario 3, los datos sensibles del pedido que salen hacia otro sistema

Mecanismo: cifrado asimétrico con rsa (en la práctica combinado con aes)

Justificación: cuando la tienda le manda los datos de un pedido a otro sistema, por ejemplo a la pasarela de pagos o a la transportadora, el problema es que no hay una clave compartida segura entre los dos, con rsa la tienda cifra con la llave pública del que recibe y solo él puede descifrarlo con su llave privada, que nunca sale de su servidor, entonces aunque alguien intercepte el mensaje no lo puede leer, como rsa es lento y tiene el límite de tamaño lo normal es cifrar el pedido con aes y con rsa solo la clave de aes, que es la idea de https, y de todas formas lo más importante es que la tienda no guarde el número completo de la tarjeta, que es lo que pide pci dss

4. Conclusiones

- El cifrado simétrico con aes quedó comprobado como reversible: el mensaje descifrado salió idéntico al original con la misma clave y el mismo iv y con una clave equivocada no se recupera nada (objetivo 1)
- La prueba de los dos usuarios con manizales2026 mostró que sin sal los hashes son iguales y con sal salen diferentes, y que password_hash hace lo mismo guardando la sal dentro del hash (objetivo 2)
- La diferencia más grande entre las herramientas es la velocidad y para qué sirve cada una: aes es rápido para datos que se leen después, rsa es lento pero no necesita compartir la clave y bcrypt es lento a propósito para las contraseñas (objetivo 3)
- Para negocioseguro queda: contraseñas con password_hash, backups con aes y la clave fuera del servidor, y datos que salen hacia otro sistema con rsa combinado con aes (objetivo 4)
- Un detalle que no esperaba es que la clave del código base tenía 31 caracteres y php no avisó, eso muestra que con criptografía no basta con que funcione, hay que revisar que los parámetros sean los correctos

5. Bibliografía

Owasp foundation, (s.f.), password storage cheat sheet, recuperado el 31 de octubre de 2026 de https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html

Owasp foundation, (s.f.), cryptographic storage cheat sheet, recuperado el 31 de octubre de 2026 de https://cheatsheetseries.owasp.org/cheatsheets/Cryptographic_Storage_Cheat_Sheet.html

Owasp foundation, (2021), a02:2021 cryptographic failures, recuperado el 31 de octubre de 2026 de https://owasp.org/Top10/A02_2021-Cryptographic_Failures/

The php group, (s.f.), openssl_encrypt, manual de php, recuperado el 31 de octubre de 2026 de https://www.php.net/manual/es/function.openssl-encrypt.php

The php group, (s.f.), password_hash, manual de php, recuperado el 31 de octubre de 2026 de https://www.php.net/manual/es/function.password-hash.php

Stallings, w, (2017), cryptography and network security: principles and practice (7.a ed), pearson

6. Anexos

Anexo a, código de equipo_cifrado.php, también va en el zip de la entrega

```
<?php
$metodo = "AES-256-CBC";

$claveDelProfe = "ClaveSecretaDe32CaracteresExact";
$clave = "NegocioSeguro2026ManizalesPablo!";

echo "Largo de la clave del codigo base: " . strlen($claveDelProfe) . " caracteres<br>";
echo "Largo de la clave que use: " . strlen($clave) . " caracteres<br><br>";

$iv = openssl_random_pseudo_bytes(openssl_cipher_iv_length($metodo));

$mensaje_original = "Pedido 1087 de pablo_nieto, tarjeta terminada en 4417, envio a Calle 23 # 21-40 Manizales";

$inicio = hrtime(true);
$texto_cifrado = openssl_encrypt($mensaje_original, $metodo, $clave, 0, $iv);
$tiempoCifrar = (hrtime(true) - $inicio) / 1000;

echo "Mensaje original: $mensaje_original <br>";
echo "IV (en hexadecimal): " . bin2hex($iv) . "<br>";
echo "Mensaje cifrado (en base64): $texto_cifrado <br><br>";

$inicio = hrtime(true);
$texto_descifrado = openssl_decrypt($texto_cifrado, $metodo, $clave, 0, $iv);
```

```
$tiempoDescifrar = (hrtime(true) - $inicio) / 1000;

echo "Mensaje descifrado: $texto_descifrado <br>";
echo "¿El descifrado es igual al original? " . ($texto_descifrado === $mensaje_original ? "SÍ" :
"NO") . "<br>";
echo "Tiempo para cifrar: " . round($tiempoCifrar, 1) . " microsegundos, para descifrar: " .
round($tiempoDescifrar, 1) . " microsegundos<br><br>";

$otroIv = openssl_random_pseudo_bytes(openssl_cipher_iv_length($metodo));
echo "=== EL MISMO MENSAJE CON OTRO IV ===<br>";
echo "Cifrado con otro IV: " . openssl_encrypt($mensaje_original, $metodo, $clave, 0, $otroIv) .
"<br><br>";

echo "=== DESCIFRAR CON UNA CLAVE EQUIVOCADA ===<br>";
$claveEquivocada = "OtraClaveQueNoEsLaDelEquipo12345";
$intento = openssl_decrypt($texto_cifrado, $metodo, $claveEquivocada, 0, $iv);
echo "Resultado: " . ($intento === false ? "falla, no se recupera el mensaje" : $intento) . "<br>";
?>
```

Anexo b, código de equipo_hash_sal.php, también va en el zip de la entrega

```
<?php
$claveUsuario1 = "manizales2026";
$claveUsuario2 = "manizales2026";

echo "Usuario 1: laura_gomez, clave: $claveUsuario1 <br>";
echo "Usuario 2: andres_rios, clave: $claveUsuario2 <br><br>";

echo "=== SIN SAL ===<br>";
echo "Hash Usuario 1: " . hash('sha256', $claveUsuario1) . "<br>";
echo "Hash Usuario 2: " . hash('sha256', $claveUsuario2) . "<br>";
echo "¿Son idénticos? " . (hash('sha256', $claveUsuario1) === hash('sha256', $claveUsuario2)
? "SÍ" : "NO") . "<br><br>";

echo "=== CON SAL ÚNICA POR USUARIO ===<br>";
$sal1 = bin2hex(random_bytes(16));
$sal2 = bin2hex(random_bytes(16));

$inicio = hrtime(true);
$hashConSal1 = hash('sha256', $sal1 . $claveUsuario1);
$tiempoSha = (hrtime(true) - $inicio) / 1000;
$hashConSal2 = hash('sha256', $sal2 . $claveUsuario2);

echo "Sal Usuario 1: $sal1 <br>";
```

```
echo "Hash Usuario 1 (con sal): $hashConSal1 <br><br>";
echo "Sal Usuario 2: $sal2 <br>";
echo "Hash Usuario 2 (con sal): $hashConSal2 <br>";
echo "¿Son idénticos? " . ($hashConSal1 === $hashConSal2 ? "SÍ" : "NO") . "<br><br>";

echo "=== COMPROBAR EL LOGIN CON LA SAL GUARDADA ===<br>";
$escrita = "manizales2026";
echo "laura_gomez escribe $escrita: " . (hash('sha256', $sal1 . $escrita) === $hashConSal1 ?
"clave correcta" : "clave incorrecta") . "<br>";
$escrita = "manizales2025";
echo "laura_gomez escribe $escrita: " . (hash('sha256', $sal1 . $escrita) === $hashConSal1 ?
"clave correcta" : "clave incorrecta") . "<br><br>";

echo "=== CON PASSWORD_HASH (BCRYPT, LA SAL VA DENTRO DEL HASH) ===<br>";
$inicio = hrtime(true);
$bcrypt1 = password_hash($claveUsuario1, PASSWORD_BCRYPT, ['cost' => 12]);
$tiempoBcrypt = (hrtime(true) - $inicio) / 1000;
$bcrypt2 = password_hash($claveUsuario2, PASSWORD_BCRYPT, ['cost' => 12]);

echo "Hash Usuario 1: $bcrypt1 <br>";
echo "Hash Usuario 2: $bcrypt2 <br>";
echo "¿Son idénticos? " . ($bcrypt1 === $bcrypt2 ? "SÍ" : "NO") . "<br>";
echo "password_verify con la clave correcta: " . (password_verify("manizales2026", $bcrypt1) ?
"true" : "false") . "<br>";
echo "password_verify con una clave equivocada: " . (password_verify("manizales2025",
$bcrypt1) ? "true" : "false") . "<br><br>";

echo "=== VELOCIDAD ===<br>";
echo "sha256 con sal: " . round($tiempoSha, 1) . " microsegundos<br>";
echo "password_hash con costo 12: " . round($tiempoBcrypt / 1000, 1) . " milisegundos<br>";
?>
```